

```

1 #=====
2 # MIT 6.046J 代码精读 • PS10 - 异步 BFS
3 # 源文件: async_bfs.py (来自 GitHub mit-6046)
4 #=====
5
6 """PS10 Problem 10-2: asynchronous BFS with NO corrections, built level-by-level.
7
8 The naive relaxation-based async BFS can make many parent corrections
9 ( $O(n^2E)$  messages,  $O(\text{diam} \cdot n \cdot d)$  time). This algorithm has the root coordinate the
10 tree LEVEL BY LEVEL using broadcast/convergecast waves, so each process sets its
11 parent exactly once (no corrections) and can output immediately.
12
13 We SIMULATE the algorithm on arbitrary connected graphs (with adversarial but
14 finite message delays) and verify:
15 * every non-root node's final `parent` gives a genuine BFS tree (its level ==
16 true BFS distance from the root);
17 * time is  $O(\text{diam}^2 \cdot d)$  and messages are bounded as analysed.
18
19 Since it is a *synchronous-round* simulation of an asynchronous algorithm with
20 the root-coordinated phase structure, correctness (a true BFS tree, no
21 corrections) is what we certify. We compare each node's tree-level to the true
22 BFS distance computed independently.
23 """
24 from __future__ import annotations
25
26 from collections import deque
27 from typing import Dict, List, Optional, Tuple
28
29
30 def true_bfs_levels(n: int, adj: List[List[int]], root: int) -> List[int]:
31     dist = [-1] * n
32     dist[root] = 0
33     q = deque([root])
34     while q:
35         u = q.popleft()
36         for v in adj[u]:
37             if dist[v] == -1:
38                 dist[v] = dist[u] + 1
39                 q.append(v)
40     return dist
41
42
43 def coordinated_bfs(n: int, adj: List[List[int]], root: int):
44     """Root-coordinated level-by-level BFS. Returns (parent, level, rounds,
45     messages). Each node's parent is set exactly once (no corrections).
46
47     Structure per level L (broadcast-convergecast wave):
48     * BROADCAST 'search' out along the tree built so far to the current
49       frontier (depth L), which then probes all neighbours;
50     * newly discovered nodes (unassigned) reply parent(True) to the first
51       prober; already-assigned nodes reply parent(False);
52     * CONVERGECAST 'done' back to the root, which then starts level L+1.
53     We simulate this deterministically and count messages/time.
54     """
55     parent: List[Optional[int]] = [None] * n
56     level = [-1] * n
57     level[root] = 0
58     parent[root] = root
59     frontier = [root]
60     rounds = 0
61     messages = 0
62     L = 0
63     while frontier:
64         # broadcast wave from root to frontier:  $O(L)$  time along tree paths
65         rounds += 2 * L + 2 # broadcast down + convergecast up (each depth L)
66         next_frontier = []
67         # each frontier node probes all neighbours
68         # deterministic tie-break: lowest-id prober wins a new node
69         claims: Dict[int, int] = {}
70         for u in sorted(frontier):
71             for v in adj[u]:
72                 messages += 1 # search probe
73                 if level[v] == -1 and v not in claims:
74                     claims[v] = u # first (lowest-id) prober claims v
75                 messages += 1 # parent(b) reply
76         for v, u in claims.items():
77             parent[v] = u
78             level[v] = L + 1
79             next_frontier.append(v)
80         frontier = sorted(next_frontier)
81         L += 1
82     return parent, level, rounds, messages
83
84
85 def verify_bfs_tree(n: int, adj: List[List[int]], root: int,

```

```
86         parent: List[Optional[int]], level: List[int]) -> bool:
87         """Verify (parent, level) is a valid BFS tree: levels equal true distances,
88         and each non-root parent is a neighbour one level up."""
89         dist = true_bfs_levels(n, adj, root)
90         if level != dist:
91             return False
92         for v in range(n):
93             if v == root:
94                 continue
95             if dist[v] == -1:
96                 continue # unreachable (won't happen for connected graphs)
97             p = parent[v]
98             if p is None:
99                 return False
100             if p not in adj[v]:
101                 return False
102             if level[p] != level[v] - 1:
103                 return False
104         return True
105
```